Software Engineering.

1. (i) Level 0 DFD is also called context-diagram.

(ii) Functional testing or behavioral testing is also known as Black-box testing.

(iii) COCOMO stands for Constructive Cost Model.

(iv) Class, interface, feature, constraint etc are the parts of a class diagram.

(v) The Incremental Model results in successive versions of the system.

(vi) Coupling between two modules is a measure of the degree of interaction.

(vii) A minimal test suite is the smallest set of test cases that achieves the required testing coverage.

(viii) Delphi cost estimation technique is an example of Expert-Based Estimation model.

(ix) UML structure diagrams show the static structure of the system.

(x) If the code of the prototype model is used in the actual project, it is called Evolutionary Prototyping.

(xi) In a DFD, if two bubbles are directly connected by a data flow arrow, then they are directly connected Processes.

(xii) $V(G) = E - N + 2$ is a formula to calculate Cyclomatic complexity.

Group - B

2. Differentiate between object-oriented and function oriented design approaches.

Feature	Object-oriented design	Function-Oriented design.
1. Basic unit	It is based on objects and classes.	It is based on functions and procedures.
2. Design approach	It follows a bottom-up approach.	It generally follows a top-down approaches.
3. Data & Functions	Data and functions are combined into a single unit called an object.	Data and functions are treated separately.
4. OOP Feature	It supports encapsulation, inheritance and polymorphism.	It does not directly support these OOP feature.
5. Data Security	Data hiding and security are easier to achieve.	Data is often shared among functions, making data protection comparatively difficult.
6. Reusability	It provides better reusability through classes and inheritance.	Reusability is mainly achieved through functions.
7. Suitability	It is suitable for large & complex systems.	It is suitable for small and procedure-oriented systems
8. Programming Languages	Examples of Languages include Java, C++ & C#	Eamples include C & Pascal

Group - C

7. a. Data Flow Diagram (DFD):

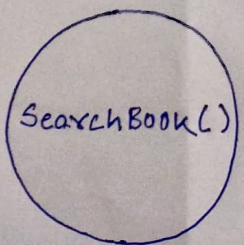
A data flow diagram is a graphical representation of the flow of data through a system. It shows how data ~~members~~ enters the system, how it is processed, where it is stored and how the output is generated.

Need of DFD:

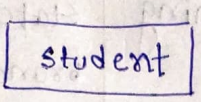
1. **Understanding the system:** It helps to understand the overall working of a system and ~~improvement~~ movement of data between diffrent components.
2. **Identifying Processes:** It identifies the major purpose that transform input data into meaningful output.
3. **Data Flow Representation:** It shows how data moves between processes, data stores and external entities.
4. **System Analysis:** It helps analysts identify missing processes, unnecessary data flows and possible system inefficiencies.
5. **Communication:** It provides a simple graphical representation that helps developers, users and clients communicate their understanding of the system.
6. **System design & documentation:** It serves as a reference for designing the system and documenting its functional requirements.

(b) Different components of DFD:

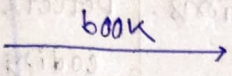
→ A function or process is represented by a bubble.



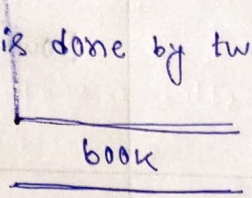
→ Entities are represented by a rectangle.



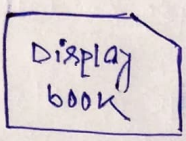
→ Data Flow is represented by an arrow.



→ Data store is done by two parallel lines.



→ Output is displayed by this symbol -



© Difference between black box testing and white box testing.

Point	Black Box Testing	White Box Testing
1. Definition	Tests the functionality of software without examining its internal code.	Tests the internal structure, logic and code of software.
2. Knowledge of code	No knowledge of internal code is required.	Knowledge of internal code is required.
3. Focus	Focus on inputs and expected outputs.	Focuses on statements, branches, path and loops.
4. Basis of testing	Based on software requirement and specification.	Based on program logic and source code.
5. Performed by	Usually performed by testers.	Usually performed by developers or testers with programming knowledge.

Point	Black-Box Testing	White Box Testing.
6. Testing techniques.	Equivalence partitioning and boundary value analysis.	Statement coverage, branch cover and path coverage.
7. Error Detection	Detects missing functions, incorrect outputs and interface errors.	Detects logical errors, coding errors and control flow errors.